

AGENT HARNESS

效率决策， 不等于质量决策

对《Your agent harness is an efficiency decision, not a quality decision》的工程化复盘、统计边界与可落地 Harness Design Playbook

核心命题

复杂 Harness 不应被默认视为“智能增益器”。它首先改变的是 trajectory、token、工具调用、handoff、验证与延迟；只有当它稳定补偿了当前模型的具体 failure mode，才有资格被视为质量组件。

Know-Why × Know-How

从实验读数，到 Agent Runtime 的设计原则、Eval 指标与 Ablation 方法

版本 v1.0 • 2026-08-14

基于 Nawk 2026-07-19 / 2026-07-27 更新文章，并结合 Anthropic 2025-2026 Harness / Eval 工程资料扩展分析。

结论先行：不要把 Harness Complexity 当成 Intelligence

这篇文章最有价值的地方，不是“证明 Harness 没用”，而是提供了一个受控视角：固定同一模型，只替换模型外部的 Harness。结果显示，在 8 个聚焦型真实 bugfix 上，四套 Harness 的质量点估计没有形成清晰分离，但 token、工具调用和墙钟时间出现了倍数级差异。

一句话沉淀

Harness 的目标不是最大化 scaffolding，而是以最小 orchestration 成本，稳定补偿当前模型真实存在的 failure modes。

- 原文支持的核心事实：Pi、OpenCode、Claude Code、Nanocoder 使用同一 DeepSeek V4 Flash；质量区间高度重叠，而 Claude Code / Nanocoder 的输出 token 与耗时显著更高。
- 最重要的评测启发：Agent Eval 的对象不是单条 trajectory，而应是同一条件下的 trajectory distribution；少量重复运行很容易把随机波动误认成 Harness 提升。
- 最重要的系统启发：Subagent、Planner、Reviewer、RAG、Memory、Graph、Ralph Loop 都应被视为“failure-mode compensator”，而不是默认必备模块。
- 最重要的优化方法：从 Baseline 出发，定位失败模式，加入最小干预，再用 repeated eval + ablation 证明收益；不能证明 load-bearing 的组件应被删除。
- 与 Anthropic 并不矛盾：在短、聚焦、已落入模型稳定能力范围的任务上，Harness 更像效率变量；在长周期、跨上下文、强验证、位于能力边界附近的任务上，Harness 仍可能显著改变质量。

MAP

报告结构

1. 原文实验：控制变量、四类 Harness、关键数据与轨迹差异
2. Know-Why：为什么复杂 Harness 容易产生 Harness Tax 与 Orchestration Tax
3. 评测边界：run-to-run variance、置信区间与“没有发现差异”
4. 统一模型：Task Difficulty × Model Capability × Harness Value
5. Ralph Loop / Graph Engineer / Subagent / Tool Design 的具体启示
6. Know-How：Minimal Sufficient Harness、Ablation、Runtime Metrics 与 Eval Blueprint
7. 落地清单：何时增加 Harness、何时删除 Harness、如何持续复验

1. 文章到底测试了什么？

作者刻意固定“脑”，只替换“外壳”：同一个本地 DeepSeek V4 Flash (High)，使用相同任务与评分方式，分别运行在 Pi、OpenCode、Claude Code 和后来补充的 Nanocoder 上。这个设计把模型能力尽量固定，使 Harness 的工具面、系统提示、委派策略、搜索/编辑节奏等成为主要变量。

Harness	工具数	固定开销 / turn	平均 steps	平均输出 tokens	平均墙钟时间	平均质量(0-3)
Pi	4	1,340	28.0	14,775	2.1 min	2.34
OpenCode	10	7,197	16.6	17,463	3.1 min	2.07
Claude Code	27	23,132	37.9	58,370	8.0 min	2.42
Nanocoder	15	6,121	37.4	55,844	5.2 min	2.25

数据来源：Nawk 文章中的任务加权平均。作者强调：四个平均质量点估计存在抖动，但 95% 置信区间相互重叠，无法在该实验规模和任务域上得到清晰的 Harness 排名。

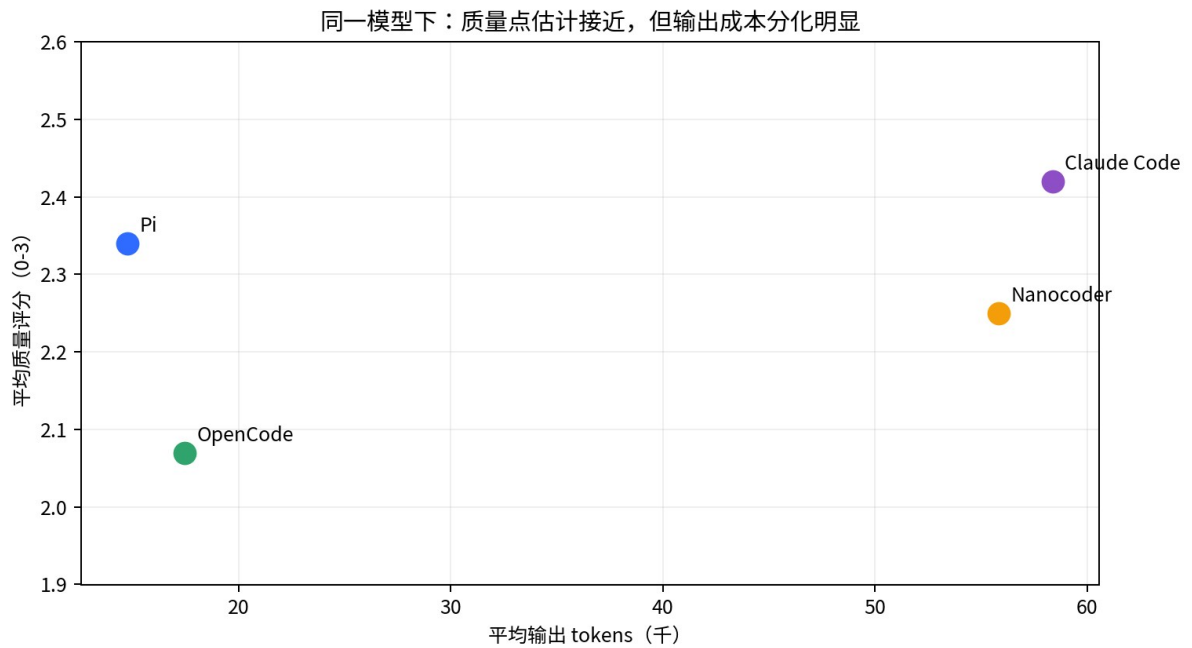


图 1 | 同一模型下，质量点估计接近，而输出 token 显著分化

Scope 很重要

原文自己的结论也明确限定在 DeepSeek V4 Flash + 8 个 focused bug fixes + large codebase 的 agentic workload。它不是“所有模型、所有任务、所有 Harness”的普遍证明。

1.1 为什么这个实验设计有价值

- 它避免把“模型更强”误记成“Harness 更强”：底层模型被固定。
- 它让效率指标成为一等公民：不仅看最终 diff，还看 output tokens、wall-clock、steps、tool calls。
- 它暴露了不同 Harness 的工作风格：single-context reasoning、subagent delegation、heavy exploration。
- 它提醒 Agent 工程：同一最终质量可以由完全不同的 trajectory 达成，而 trajectory 本身决定成本、延迟与可观测性。

PART I • SOURCE-DERIVED

2. 最关键结果：质量没有清晰拉开，效率却拉开了倍数

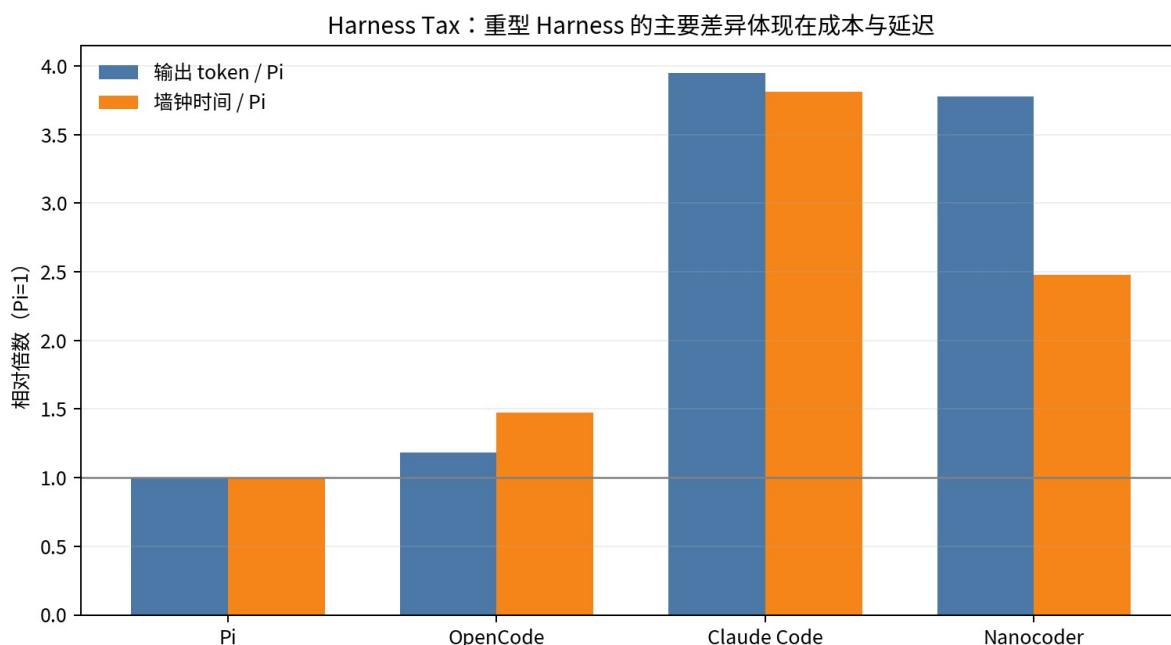


图 2 | 以 Pi=1 归一化：重型 Harness 的 token 与耗时成本显著增加

按文章表格，Claude Code 的平均输出 token 约为 Pi 的 3.95 倍，平均墙钟时间约为 3.81 倍；Nanocoder 约为 Pi 的 3.78 倍输出 token 和 2.48 倍墙钟时间。与此同时，四者的平均质量点估计都落在 2.07-2.42 的窄区间。

不要误读

这不等于“质量完全相同”或“Harness 对质量永远无效”。更准确的表述是：在该任务集与样本量下，没有观察到足以从 run-to-run noise 中稳定分离出来的质量差异。

2.1 Pi : reasoning-heavy

Pi 的核心特征是薄：4 个工具、较短固定上下文，主要在单个 context 中完成读、想、改、测。它把更多预算用在连续推理，而不是 orchestration。

2.2 OpenCode : delegation-heavy

OpenCode 的主 trajectory 看起来很短，但文章指出：如果只统计主 Agent，会低估真实成本。把 subagent 的输出 token 加回后，它从表面约 11K 变成约 17K，接近 Pi 的约 15K。差异主要转移到 handoff 和 tool-running 的时间上。

2.3 Claude Code / Nanocoder : exploration-heavy

两者更倾向先“看足够多”再编辑。原文报告 Claude Code 平均约 70 次工具调用/任务，Pi 约 40，OpenCode 约 22；Claude Code 第一次 edit 往往出现在 trajectory 的约 80% 位置，Nanocoder 约 70%。这些前置工作大多是 read / search / re-read。

典型 Trajectory Policy：差异不只是“工具多少”，而是动作序列与停止策略

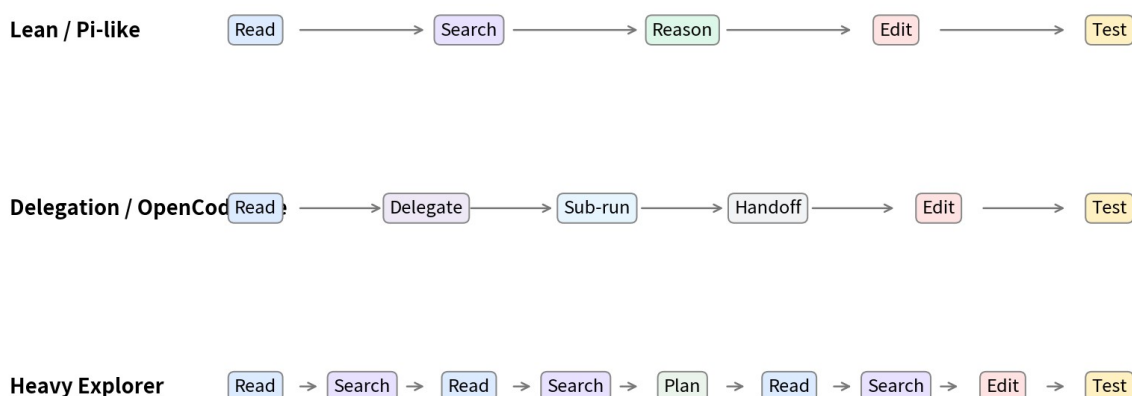


图 3 | 不同 Harness 的差异更像 trajectory policy，而不只是工具数量

3. 真正重要的发现：Agent Eval 必须评估分布，而不是一次运行

文章指出，在较难任务上，同一 Harness 仅因随机性就可能从 “basic attempt” 摆动到 “solid fix”，相差一个完整 grade。Pi、OpenCode、Claude Code 各有 24 次运行（每个任务 3 次），Nanocoder 有 64 次运行（每任务 8 次）。这说明 Agent 的 stochasticity 足以吞掉小幅点估计差异。

核心原则

不要 eval 一条 trajectory；要 eval 一个 trajectory distribution。

工程上至少需要同时关注以下统计对象：

维度	建议关注
中心趋势	E[quality]、pass rate、complete rate
不确定性	variance、confidence interval、不同 seed 波动
风险尾部	catastrophic failure rate、regression rate
成本	tokens、tool calls、\$ / task、\$ / successful task
时延	time-to-first-action、time-to-first-edit、time-to-done

3.1 为什么 n=1 / n=3 很危险

当 effect size 小、随机波动大时，少量运行会把 seed luck、基础设施噪声、偶发工具失败与真实架构差异混在一起。于是“这个 Prompt 提升 20%”“这个 Harness 强 15%”可能只是一次抽样结果。

原文给出的“低于约 10 次可能主要测到天气”应理解为工程经验警告，而不是统计学上的通用阈值。实际所需 trial 数取决于任务难度、评分方差、预期 effect size 和你要达到的置信度。

3.2 界：CI 重叠 ≠ 证明等价

严格来说，两个 95% confidence interval 重叠，并不能直接推出“两个 Harness 相等”；同样，“未能拒绝差异为 0”也不等于“已经证明差异足够小”。如果目标是真正证明工程等价，更适合预先定义一个可接受差异 ϵ ，并做 equivalence / non-inferiority 风格的分析。

更严谨的结论语言

“没有观察到清晰且稳定的 quality separation” 比 “Harness 对质量没有影响” 更精确。

PART II • KNOW-WHY

4. Harness Tax：复杂性到底花在哪里？

Harness 的成本不是一个数字，而是一组叠加税。可以把总延迟近似分解为：

Latency Decomposition

$$L_{\text{total}} = L_{\text{generation}} + L_{\text{tool}} + L_{\text{handoff}} + L_{\text{queue}} + L_{\text{orchestration}} + L_{\text{recovery}}$$

对于 Multi-Agent，真实 token 成本也不能只统计主 Agent：

Token Accounting

$$T_{\text{total}} = T_{\text{main}} + \sum T_{\text{subagent}} + T_{\text{tool_results}} + T_{\text{reconstruction}}$$

- Subagent spawn：启动独立推理上下文、分配任务、生成结果。
- Context reconstruction：下游 Agent 必须重新理解任务、状态和上游产物。
- Handoff serialization：中间状态需要写入消息、文件或结构化 artifact。
- Tool scheduling：并行/串行工具执行、等待、失败重试都会产生墙钟成本。
- Orchestration reasoning：主 Agent 需要决定“谁来做”“何时合并”“是否继续”。

4.1 Token Accounting Illusion

如果只看主 Agent，Multi-Agent 很容易制造“更省 token”的错觉。主 Agent 8K + 三个子 Agent 12K/7K/6K，真实总成本不是 8K，而是 33K；如果还加上上下文重建和工具结果，会更高。

4.2 每一个 Agent Boundary 都是 Context Boundary

Planner→Coder、Coder→Reviewer、Reviewer→Fixer 每跨一次边界，都要支付信息压缩、丢失、歧义和重建成本。因此不要问“这里能不能再加一个 Agent”，而应该问“这里是否存在稳定、可观测、单 Agent 解决不好的 failure mode，足以支付这次 boundary 的成本”。

5. Epistemic Over-Search：为什么“更勤奋”可能只是更慢

Claude Code / Nanocoder 在文章中呈现出一个典型 failure mode：大量 read/search/re-read 发生在 edit 之前，最后仍只修改少量代码。问题不是“搜索本身无用”，而是缺少明确的 stopping criterion。

边际信息增益视角

当 $\text{Information Gain}_t \rightarrow 0$ ，但 $\text{Cost}_t > 0$ ，而 Agent 仍持续搜索时，就进入 epistemic over-search。

一个更好的策略不是“尽可能彻底地调查”，而是让每次 tool call 都对应一个明确的不确定性：它在验证什么 hypothesis？如果这次观察不会改变下一步动作，是否应该停止搜索并进入 edit / test？

5.1 TTFE：Time-to-First-Edit

这篇文章自然引出了一个实用诊断指标：Time-to-First-Edit（或 Tool Calls Before First Edit）。TTFE 太低可能代表 premature action，太高可能代表 analysis paralysis / tool thrashing。真正要优化的是“足够理解 → 尽快行动 → 用测试校正”的中间区域。

模式	典型轨迹	风险
Premature Action	Read → Edit	局部理解不足，容易修错层或引入回归
Healthy Loop	Read → Hypothesis → Targeted Search → Edit → Test	较低成本获得可验证进展
Analysis Paralysis	Read/Search × N → Edit	边际信息增益下降，成本继续上升

5.2 Think or Observe?

Agent runtime 中经常存在一个被忽略的决策：当前不确定性到底应该由内部 reasoning 解决，还是必须调用外部工具获得新事实？如果模型已经拥有所有关键代码事实，继续读 10 个文件可能不如认真推演一次因果链；反之，如果缺少真实状态，纯思考只会幻觉。

6. 真正优化对象不是 Tool 数量，而是 Trajectory Policy

“Pi 只有 4 个工具，Claude Code 有 27 个工具”很容易被误读成“工具越少越好”。更深层的变量是策略：在当前 state 下，Agent 如何选择 read / search / bash / edit / delegate / plan / test，以及何时停止。

Policy View

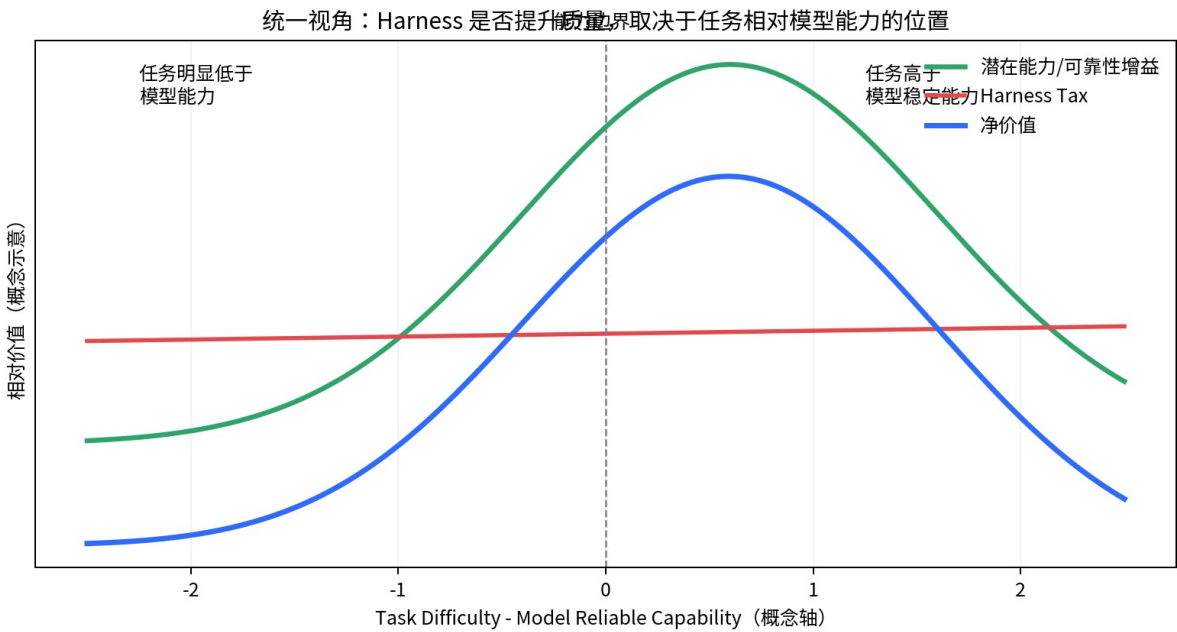
Harness Optimization $\approx$ optimize $P(\text{next_action} \mid \text{state, tools, prompt, history, uncertainty})$

- 工具面越大，selection entropy 往往越高，但不是必然低效。
- 真正危险的是“大工具面 + 探索偏置 + 不明确停止规则 + 高不确定性”，它会诱导更长 trajectory。
- 因此 Tool Design 的目标不是简单砍数量，而是让默认工具面足够小、语义边界清晰、按需渐进披露。
- 在工具调用可被代码执行/批处理合并时，应减少“模型每一步都来回调度”的 round-trip。

PART III • SYNTHESIS

7. 与 Anthropic 为什么并不矛盾：关键是 Capability Gap

Anthropic 2026 年的 long-running harness 文章展示了相反的现象：在一个复杂全栈应用生成任务上，完整 Harness 从 20 分钟 / \$9 的 solo run 增长到约 6 小时 / \$200，但作者观察到核心功能和整体质量的明显提升。随后在 Opus 4.6 上，他们又删除了部分原先 load-bearing 的 scaffolding，并明确指出：当任务落入新模型的稳定能力范围时 evaluator 会变成 unnecessary overhead；当任务仍位于能力边界时 evaluator 继续提供真实 lift。



这两组材料可以统一成一个变量：Gap = Task Difficulty - Model Reliable Capability。

区域	D-C	典型任务	Harness 预期作用
A	明显 < 0	短 bugfix、范围清晰、单 repo、短 horizon	质量增益小；复杂 Harness 更可能成为成本
B	≈ 0	复杂功能、跨层验证、长 trajectory、边界能力	Planner / Evaluator / structured state 可能显著提

			升可靠性
C	明显 > 0	远超模型知识或推理能力的任务	Harness 可组织失败，但不能凭空制造缺失智能

统一规律

Harness 的 load-bearing components 会随着模型能力移动。一个在 Sonnet 4.5 上必要的 context reset，到了更强模型可能成为 dead weight；一个今天必要的 evaluator，明天也可能只剩 overhead。

PART III • APPLICATION

8. 对 Ralph Loop 的启示：它首先是 Horizon Amplifier

Ralph Loop 的核心价值在于“持续推进”：把一次可能过早终止的 Agent trajectory 变成可重复 inspect → work → verify → continue 的闭环。它主要补偿 premature termination、跨阶段持续性和长周期任务中的进度遗忘，而不是让每一步瞬间更聪明。

适合 Ralph Loop	不应默认使用
任务需要跨多轮推进、状态可持久化、存在客观 done condition	两分钟即可完成的小修复，却强制多轮 plan/review/checkpoint
模型容易提前收尾，必须持续迭代到测试通过	没有清晰 stopping criterion，只是“再循环一次看看”
每轮可产生可验证的新状态	每轮只是重复阅读/总结，没有状态改进

Ralph 设计原则

Loop 必须有“progress signal”和“termination rule”。没有这两个条件，loop 很容易把 persistence 变成 cost amplification。

9. 对 Graph Engineer 的启示：每条 Edge 都要证明价值

Graph 很适合表达确定的流程、权限、并行性与检查点，但图结构的“漂亮”并不等于输出质量提升。每一个节点意味着一次独立策略，每一条 edge 往往意味着一次状态传递。

Graph Cost

$$C_{\text{graph}} \approx \sum \text{NodeCompute} + \sum \text{Handoff} + \sum \text{ContextReconstruction} + \text{CoordinationOverhead}$$

Graph Engineer 应该为每个节点写出“它防止哪个 failure mode”，并为每条边写出“为什么这里需要 context boundary”。如果不能回答，就应尝试把节点折回主 Agent 或改为普通工具/函数。

PART III • DESIGN PRINCIPLE

10. 把所有 Harness 组件重定义为 Failure-Mode Compensator

组件	它应该解决的具体 failure mode	常见过度使用
Planner	under-spec、直接开工导致范围不足	简单任务也强制长计划
Evaluator / Reviewer	自评偏乐观、漏测、最后一公里缺陷	模型已能稳定自检却仍每步复审
Subagent	可并行的独立子问题、专业化上下文	把可在一个 context 内完成的工作拆碎
Memory / Handoff	跨 session 遗忘、长 horizon 状态丢失	短任务也写大量状态文件
Context Reset	context anxiety、历史污染	模型已不受影响仍定期清空
RAG / Search	事实/代码定位失败	不设 stop rule 的无边界检索
Ralph Loop	过早终止、需要持续进展	没有 progress signal 的死循环
Graph	确定流程、权限边界、并行/汇合	为了“架构感”增加节点
Sandbox / Permission	隔离不可信执行、限制副作用	这是安全/可靠性组件，不能只按 token ROI 删除

注意：安全类 Harness 组件（sandbox、permissions、credential isolation）不应只按“是否提高任务评分”来评估。它们优化的是 blast radius、合规性与可恢复性，价值函数不同。

PART IV • KNOW-HOW

11. Minimal Sufficient Harness：正确的开发顺序

核心方法

Baseline → observe failure → add minimum intervention → repeated eval → ablation → keep/remove

- 8. 从最薄、可工作的 Baseline 开始，而不是先搭全套 Planner/Reviewer/Memory/Subagents。
- 9. 在真实任务上读 traces，给失败模式命名：定位失败？过早编辑？过度搜索？漏测试？跨窗口遗忘？
- 10. 只加入一个最小机制，明确它预期改变的指标和 failure mode。

- 11. 对同一任务集合重复运行，比较 quality distribution、cost、latency、variance，而不是只看 best run。
- 12. 做 ablation：把新增组件关掉，验证质量/可靠性是否真的下降。
- 13. 模型升级、任务分布变化后重新 ablate；历史上有效不代表今天仍 load-bearing。

11.1 一个实用的决策门

问题	YES	NO
是否存在稳定、可复现的 failure mode?	继续	先补 eval / trace，不要加组件
这个 failure mode 能被更简单的 prompt/tool/schema 修复吗?	先用更简单方案	继续
新组件是否产生可测量 lift?	继续	删除
lift 是否大于 token/latency/complexity tax?	保留并监控	回滚或改轻量实现
模型升级后仍然 load-bearing?	保留	删除/降级为按需启用

PART IV • KNOW-HOW

12. Harness Eval Blueprint：怎么真正比较两个 Harness

Anthropic 的 eval 指南建议把 coding agent 的最终 outcome 检查与 transcript 指标组合起来。对于 Harness 对比，建议至少分成 Quality、Trajectory、Cost、Latency、Risk 五层。

层	建议指标
Quality	fail-to-pass tests、pass-to-pass tests、functional outcome、LLM rubric（辅助）
Trajectory	n_turns、n_toolcalls、read/search/edit/test 比例、重复读取率、TTFE
Cost	input/output tokens、主/子 Agent 分账、tool-result tokens、\$ / run
Latency	generation、tool execution、handoff、queue、time-to-first-edit、time-to-done
Risk	regression、catastrophic failure、权限违规、不可恢复

状态、variance

12.1 推荐的最小实验设计

14. 固定模型、temperature/seed 策略、环境、repo revision、任务描述、grader。
15. Harness A/B 都运行相同 task bank；对高方差任务增加 trials。
16. 不要只报告均值；同时报告分布、置信区间、成功率和失败类型。
17. 把 subagent 与后台工具的 token / latency 全部计入总账。
18. 至少抽样阅读 traces，检查 grader 是否奖励了“正确但不同”的解法，或惩罚了 harmless variation。
19. 做 component-level ablation，而不是只做整个 Harness A vs B。

12.2 一个可复用的 Harness Scorecard

指标	定义 / 公式	为什么重要
Quality Mean	mean(task score)	看平均能力
Quality Var	variance(score)	看稳定性，不被单次 demo 欺骗
Success Rate	successful trials / all trials	比均值更直观
Cost / Success	total cost / successful trials	把成本与产出绑定
Time / Success	total wall time / successful trials	衡量工程吞吐
TTFE	time from start to first edit/action	诊断 over-search / premature action
Repeated Read Ratio	re-read calls / read calls	诊断工具抖动
Tool Utility	useful calls / exposed-or-called tools	衡量工具面是否有效

PART IV • RUNTIME

13. Agent Runtime 应该沉淀哪些可观测数据

如果未来要持续优化 Harness，trace schema 必须从第一天就支持“归因”。否则只知道任务失败，却不知道是模型、工具、context、handoff 还是 evaluator 的问题。

域	建议字段
Run	run_id, task_id, harness_version, model_version, seed, repo_sha

LLM	turn_id, role, input_tokens, output_tokens, cache, latency
Tool	tool_name, args_hash, start/end, status, result_size, retry_count
Agent	agent_id, parent_id, spawn_reason, handoff_size, subagent_tokens
State	first_edit_at, tests_at, checkpoint_count, context_reset_count
Outcome	grader scores, tests passed, regression, human override

关键要求

不要让“主 Agent trace”冒充“完整系统 trace”。任何 subagent、后台 evaluator、自动工具循环都必须进入统一 accounting。

PART IV • DECISION PLAYBOOK

14. 何时应该加 Harness？何时应该删 Harness？

观察到的现象	优先尝试	避免第一反应
模型直接开工、明显 under-scope	轻量 spec/planner 或先写 acceptance criteria	立刻上多 Agent graph
模型做完后漏关键功能	deterministic tests + targeted evaluator	每一步都 review
长任务后半段失去目标	structured state / checkpoint / memory	无限扩大 context
搜索很多仍不编辑	hypothesis-driven search + stop rule + TTFE budget	再加更多 search tools
工具选择混乱	缩小默认 tool surface / namespace / progressive disclosure	堆更多同义工具
任务可并行且相互独立	subagents with explicit contracts	为了“并行”拆有强依赖的步骤
模型已经稳定完成某组件负责的任务	ablate / remove / on-demand enable	因为历史有效就永久保留

PART IV • ANTI-PATTERNS

15. Harness Engineering 的 8 个常见反模式

- 20. Feature Accretion: Agent 不够好就继续加 Planner、Memory、Reviewer、RAG，却没有 failure attribution。
- 21. Demo-Driven Evaluation: 用 1-3 个漂亮 demo 宣布架构提升。
- 22. Main-Agent Accounting: 只算主 Agent，不算 subagent、evaluator、handoff。
- 23. Tool Surface Inflation: 工具越多越觉得强，忽略 selection entropy 与 schema overhead。
- 24. Over-Search by Prompt: 用 “thoroughly investigate” 代替 hypothesis 与 stopping rule。
- 25. Graph for Architecture’s Sake: 节点来自组织图而不是 failure mode。
- 26. Permanent Scaffolding: 模型升级后不重新 ablate，旧补丁变成 dead weight。
- 27. Quality-Only Optimization: 忽略 latency、variance、cost per successful task 与安全性。

PART V • DISTILLATION

16. 最终沉淀：10 条 Harness Engineering 原则

- 1 复杂度不是能力** Harness complexity 本身不构成 intelligence；它必须通过可测的 failure-mode reduction 证明价值。
- 2 先测 Baseline** 不知道最薄系统能做到什么，就无法知道任何新增组件的真实 lift。
- 3 评估分布** Agent 是 stochastic system；比较的是 trial distribution，不是单次 trajectory。
- 4 完整记账** 主 Agent、subagent、tool、handoff、evaluator 都进入 token 与 latency 总账。
- 5 优化策略** 核心对象是 trajectory policy：何时观察、思考、行动、验证与停止。
- 6 搜索必须有问题** 每一次检索都应减少一个明确 uncertainty；边际信息增益接近 0 时停止。
- 7 边界昂贵** 每一个 agent boundary 都是 context boundary；先证明需要，再拆。
- 8 能力边界会移动** 模型升级会让旧 Harness 组件失去 load-bearing 性；持续 ablation。

- 9

按任务启用 Evaluator、Subagent、RAG、Graph 最好按 difficulty / uncertainty / horizon 动态启用。
- 10

追求最小充分 目标是 Minimal Sufficient Harness，而不是 Maximum Possible Harness。

PART V • CHECKLIST

17. 可直接用于项目评审的 Harness Checklist

检查项	通过标准
Failure Mode	能用一句话描述该组件在防什么失败
Evidence	至少有一组 repeated eval 显示可测 lift
Ablation	关闭组件后，关键指标确实恶化
Accounting	已计入全部 subagent/tool/evaluator 成本
Stopping	搜索、循环、review 有明确停止条件
Difficulty Gate	组件可按任务难度/不确定性按需启用
Model Upgrade	每次模型升级重新跑 ablation
Safety Exception	安全隔离/权限组件不以单纯 quality ROI 删除
Observability	trace 能归因到具体节点、工具、handoff
Simplicity	不存在更简单机制达到相同目标

最终判断

如果一个 Harness 组件既不能明确指出 failure mode，也不能通过 ablation 证明 lift，那么它更可能是架构装饰，而不是 load-bearing engineering。

APPENDIX A

附录 A：原文事实与扩展推导的边界

内容	性质	说明
四 Harness 的工具数、固定开销、steps、tokens、wall time、平均质量	原文事实	来自 Nawk 文章表格
Claude Code 约 70 tool calls; first edit 约 80%	原文事实	来自 Nawk 的 trajectory 分析
OpenCode 计入 subagent 后约 17K tokens	原文事实	来自 Nawk 对 token accounting 的说明
“trajectory distribution” 评测原则	工程综合	由原文 run-to-run variance + Anthropic eval 资料抽象
Harness Tax / Orchestration Tax 公式	工程模型	用于归因的概念分解，不是原文公式
TTFE、Repeated Read Ratio、Tool Utility	扩展指标	基于原文轨迹现象提出的 runtime 指标
Capability Gap 三区域模型	综合推导	结合 Nawk 与 Anthropic 2026 harness 迭代经验
Minimal Sufficient Harness	方法论沉淀	由 ablation、简化原则与工程实践综合

APPENDIX B

附录 B：一个 Harness Ablation 实验模板

下面是一个可直接复制进工程评审文档的实验结构：

字段	示例
Hypothesis	Evaluator 能降低 “功能看似完成但核心交互未实现” 的失败率
Baseline	单 Agent + deterministic tests
Treatment	Baseline + evaluator（仅在最终阶段）
Task Bank	30 个真实历史失败任务，按难度分层
Trials	每个条件每任务 5-10 次，难题增加 trials

Primary Metric	functional success rate
Secondary	quality rubric、 regression、 variance
Efficiency	output tokens、 tool calls、 wall time、 cost/success
Ablation	关闭 evaluator；同时对 evaluator prompt 做轻/重两档
Decision Rule	质量/可靠性 lift > 预设阈值且 cost/success 可接受才保留

REFERENCES

参考资料

[1] Nawk. “Your agent harness is an efficiency decision, not a quality decision.” 2026-07-19, updated 2026-07-27. <https://nqawhc.github.io/articles/harness-efficiency-not-quality/>

[2] Anthropic. “Harness design for long-running application development.” 2026-03-24. <https://www.anthropic.com/engineering/harness-design-long-running-apps>

[3] Anthropic. “Scaling Managed Agents: Decoupling the brain from the hands.” 2026-04-08. <https://www.anthropic.com/engineering/managed-agents>

[4] Anthropic. “Demystifying evals for AI agents.” 2026-01-09. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

[5] Schenker, N.; Gentleman, J. F. “On Judging the Significance of Differences by Examining the Overlap Between Confidence Intervals.” The American Statistician, 2001, 55(3):182-186.

阅读建议

如果只读一遍：先看第 2、3、7、11、12、16 章。它们分别回答“发生了什么、为什么不能信单次结果、何时 Harness 真正提升质量、怎么构建、怎么验证、最后应记住什么”。